

CAP Theorem In Database Designing

CAP theorem

It states that it is not possible to guarantee all three of the desirable properties – consistency, availability, and partition tolerance at the same time in a distributed system with data replication.

1. CP database

A CP database prioritizes Consistency and Partition Tolerance from the CAP theorem. This means:

- **Consistency:** All users see the same data, even after updates. If one user updates the database, everyone else will see the updated value immediately.
- **Partition Tolerance:** The database continues to work even if there is a network failure or a part of the system is unreachable.

However, it sacrifices Availability, meaning the system might not respond during network issues to maintain data accuracy.

Example:

Banking systems use CP databases because ensuring accurate account balances is more critical than being always available.

2. AP database

An AP database is a type of database that prioritizes Availability and Partition Tolerance from the CAP theorem.

- **Availability:** The database ensures that every request (read or write) gets a response, even if some parts of the system are down.
- **Partition Tolerance:** The database continues to work and provide responses even if there is a network partition (communication break between different parts of the system).

AP databases may not guarantee Consistency (in the strictest sense), meaning different nodes might have slightly different data for a short time.

Example:

Cassandra, In this system, the focus is on ensuring that the database can always respond to requests, even if some parts of the system are temporarily unavailable or can't communicate with each other.

3. CA Database

A CA database is a type of database that prioritizes Consistency and Availability but does not guarantee Partition Tolerance.

- **Consistency** means that every read from the database returns the most recent write. All users see the same data at the same time.
- **Availability** means that the database is always available to respond to queries, even if some parts of the system fail.

However, Partition Tolerance is sacrificed in a CA database. This means that if there is a network issue, the database might stop functioning rather than returning inconsistent or unavailable data.

Example:

CA databases are ideal when network partitioning is not a common concern, such as in smaller, local systems where quick, consistent access to data is more important than handling major network failures.

How to select the right database?

Choosing the right database depends on the needs of your application. Here are a few key factors to consider when making this decision:

1. Data Structure:

- **Relational Databases (SQL):** If your data is structured, and you need to handle complex relationships
- **Non-Relational Databases (NoSQL):** If your data is unstructured or semi-structured.

2. Scalability Needs:

- **Relational Databases** typically scale vertically (adding more power to a single server).
- **Non-Relational Databases** often scale horizontally (adding more servers to distribute the load).

3. Consistency vs. Availability:

- If your application requires strong consistency go for a relational database.
- If your app needs to be highly available and can tolerate some inconsistency for a short time, a NoSQL database may be more suitable.

4. Transaction Support:

- If you need ACID properties (Atomicity, Consistency, Isolation, Durability) for transactions, a relational database is the best option.
- If your system can work without strict transaction guarantees, NoSQL databases offer flexibility and speed.

5. Development Speed & Flexibility:

- **Relational Databases** have a predefined schema, so they're best when you need a stable, structured design.
- **NoSQL Databases** offer more flexibility, so they're better suited for projects that evolve rapidly or need to handle changing types of data.